descent, or one of the other categories of algorithms we'll discuss later in the book.

$$J(V) = \delta \cdot \sum_{v \epsilon V} \sum_{u \epsilon V/\{v\}} \|u - v\|_2^2 - \beta \cdot \sum_{v \epsilon V} \sum_{u \epsilon \text{adj}(v)} \|u - v\|_2^2$$

where the term inside the summations is the squared 2-norm, that when computed on the (vector) difference of the two points, gives us the square of the distance between the two points.

$$\|u - v\|_2^2 = (u_x - v_x)^2 + (u_y - v_y)^2$$

If you are wondering why we use the squared distance, it's not just because it's cheaper to compute[20]: the main reason is that the derivative of a square root is a pain. And, of course, the shape of the function's surface would also be different.

Now, that's a big improvement with respect to a step function: this function is at least differentiable, and that's pretty good already – partial derivatives with respect to x and y coordinates of a generic vertex w can be precisely computed:

$$\frac{\partial}{\partial w_x} J(V) = -4 \cdot \delta \cdot \sum_{u \epsilon V/\{w\}} (u_x - w_x) + 4 \cdot \beta \cdot \sum_{u \epsilon \text{adj}(w)} (u_x - w_x)$$
$$\frac{\partial}{\partial w_y} J(V) = -4 \cdot \delta \cdot \sum_{u \epsilon V/\{w\}} (u_y - w_y) + 4 \cdot \beta \cdot \sum_{u \epsilon \text{adj}(w)} (u_y - w_y)$$

Scalars β and δ are so-called hyper-parameters of the algorithm: they balance the importance of the attractive and repulsive force, and we need to adjust their values to get the result we want, and this can be done manually, or automatically.

This isn't always easy, of course: for instance, a large value for the attractive force parameter will work well for sparse graphs,